Reproducing LoRA on GLUE

CS 5782 Final Project Group 54
Yifei Zhang (yz3534), Kahei Lam (kl2235)
https://github.com/althealam/CS5782-LoRA-Final-Project

1. Introduction

Fine-tuning every weight of a large pretrained model for each downstream task is costly. *LoRA: Low-Rank Adaptation of Large Language Models* (Hu et al., ICLR 2022) [1] freezes the pretrained weights W_0 and learns a low-rank residual $\Delta W = \frac{\alpha}{r} BA$ with $A \in R^{r\times d}$, $B \in R^{k\times r}$, $r \ll \min(d, k)$, so that

$$h = W_0 x + \tfrac{\alpha}{r} BA\, x.$$

Key contributions of the paper:

- Achieve full-fine-tuning quality with $\leq 1\%$ trainable parameters on GLUE.
- Show that the update ΔW has low intrinsic rank (small r already works).
- Adapter weights can be *merged* into W_0, giving zero inference overhead.

2. Chosen Result

We reproduce the GLUE accuracy claim from Table 2 of the paper: LoRA matches full fine-tuning while training $\sim$1% of the parameters. Beyond accuracy, we extend the study with three ablations on MNLI that the original paper either reports separately (rank, target modules) or leaves implicit (scaling factor α, merge equivalence). This combination directly tests *why* LoRA works, not only *whether* it works.

3. Methodology

3.1. Backbone and LoRA Implementation

We use `roberta-base` (124.6M params). Our implementation (`src/lora.py`) defines a `LoRALinear` module and recursively replaces selected linear projections in each Transformer block. The classifier head from Hugging Face is kept unchanged. During LoRA training, only the LoRA matrices and the classifier head are trainable.

3.2. Compared Methods

Full fine-tuning updates all weights. **Frozen backbone** updates only the classifier head (lower bound). **LoRA** updates the head and the low-rank adapters.

3.3. Datasets and Metrics

Six GLUE [2] tasks: MNLI, MRPC, QNLI, QQP, SST-2, CoLA. We log validation accuracy and F1, plus trainable parameter count, total parameter count, runtime, and (for CoLA) Matthew's correlation. Tokenization uses the matching Hugging Face tokenizer; max length 128.

3.4. Training Setup

Default LoRA configuration: r=8, α=16, batch size 16, 3 epochs, AdamW with linear decay, target modules {`query`, `value`}. Same epoch budget across methods to keep comparisons fair under our compute budget. Each MNLI ablation cell is one seed.

3.5. Ablations on MNLI

We sweep (i) rank $r \in \{4, 8, 16, 32\}$, (ii) scaling $\alpha \in \{8, 16, 32, 64\}$, (iii) target modules in {`Q`, `Q+V`, `Q+V+K+Dense`}, and (iv) compare merged vs. unmerged adapters at evaluation time.

4. Results and Analysis

4.1. Main GLUE Comparison

Task	Full	LoRA	Frozen
MNLI	**85.98**	84.95	53.34
MRPC	87.25	**87.75**	68.38
QNLI	**91.09**	90.48	66.30
QQP	80.41	**87.36**	75.14
SST-2	91.97	**93.46**	80.62
CoLA	**82.65**	79.39	69.13
Trainable params	124.6M	0.89M	0.59M
% of full	100%	0.71%	0.47%

Table 1: GLUE validation accuracy (%) and trainable-parameter cost.

LoRA stays within ∼1 point of full fine-tuning on MNLI, QNLI, and CoLA, and exceeds it on MRPC, QQP, SST-2, while training ∼140× fewer parameters. Frozen backbone collapses on MNLI/QNLI, confirming that the bulk of task adaptation must happen inside the backbone, not just at the head.

4.2. Ablations on MNLI

(a) Rank, with α=16, targets=Q+V				
r	4	8	16	32
Acc	85.73	85.66	**85.94**	85.63
(b) Scaling α, with r=8, targets=Q+V				
α	8	16	32	64
Acc	85.03	85.66	85.79	**86.28**
(c) Target modules, with r=8, α=16				
Targets	Q	Q+V	Q+V+K+Dense	
Acc	81.43	84.92	**85.92**	
Params	0.74M	0.89M	1.18M	

Table 2: MNLI validation accuracy under three ablations.

Three observations: (i) accuracy is nearly flat across $r \in \{4, 8, 16, 32\}$, supporting the paper's claim that the update has low intrinsic rank; (ii) the scaling factor α shows a clearer monotonic trend, suggesting it acts as an effective learning-rate knob and is the most influential single hyperparameter in our sweep; (iii) adapting more attention projections helps, but with diminishing returns—`Q+V` captures most of the gain at ∼0.71% of params.

4.3. Merge Consistency

After training with r=8, α=16 on MNLI, evaluating the unmerged checkpoint and the merged-weights model gives identical metrics: accuracy 0.8566, F1 0.8557, loss 0.3767. This empirically validates that LoRA can be merged into W_0 at deployment time without altering predictions.

4.4. Discrepancies and Limitations

LoRA exceeds our full-FT runs on QQP/SST-2/MRPC. We attribute this to lack of per-task LR/epoch tuning for full fine-tuning rather than a real inversion of the paper's findings; The main discrepancy we observed was the LoRA result on MRPC: under our initial default setting, MRPC accuracy was much lower than reported in the original paper. Since MRPC is a relatively small dataset, we suspect the model was underfitting. After increasing the number of training epochs to 30 to better match the paper's setting, the result improved substantially and became much closer to the original performance. This suggests that the mismatch was caused more by training configuration than by the LoRA method itself.

Our ablations use a single seed per cell, so ≤ 1-point gaps should not be over-interpreted. Despite these caveats, every qualitative claim in the paper that we tested holds in our setup.

Method	MNLI	MRPC	QNLI	QQP	SST-2	CoLA
Frozen Backbone	53.34	68.38	66.30	75.14	80.62	69.13
Full Fine-tuning	**85.98**	87.25	**91.09**	80.41	91.97	**82.64**
LoRA	84.95	**87.75**	90.48	**87.36**	**93.46**	79.39

Table 1: Performance comparison of Frozen Backbone, Full Fine-tuning, and LoRA on GLUE tasks.

Figure 1: GLUE accuracy: full FT vs. LoRA vs. frozen backbone.

5. Reflections

Takeaways. (1) The *position* of adapters (which projections are adapted) matters more than the nominal rank in our sweep. (2) The effective scale α/r behaves like a learning-rate multiplier and dominates accuracy gains. (3) Merging is exact in practice, so the parameter savings translate directly to deployment savings.

Future work. Multi-seed runs and per-task LR tuning to strengthen baselines; scaling to RoBERTa-large or decoder-only LLMs; comparison with other PEFT methods (adapters, prefix tuning, $(IA)^3$, DoRA); and a memory/throughput study, since trainable-parameter count is only one axis of efficiency.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.

[2] A. Wang et al. *GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding*. ICLR, 2019.

[3] Y. Liu et al. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692, 2019.